EXPERIMENT - 14

Pro-Code Extension Using Custom Functions

Library Book Issue Management

Student Name	CALEB D
Register / Roll No.	241401016
Subject	Low-Code Application Development using Zoho Creator
Application	Library Book Issue Management
Platform	Zoho Creator
Record Date	16 October 2026

Aim

To extend Library Book Issue Management with a custom function, validate its inputs, return a structured result and test normal, invalid and empty-result cases.

Experiment overview

Implementation and execution of a reusable, typed Deluge function that returns validated library queue statistics.

Department of Computer Science and Business Systems

Rajalakshmi Engineering College

1. Extension Choice

The syllabus permits Zoho Catalyst / custom functions. This experiment uses a custom Deluge function in Creator, keeping the calculation close to the request records and avoiding a separate service deployment.

2. Function Contract

Property	Definition
Function name	Get_Library_Queue_Summary
Namespace / language	Default / Deluge
Signature	map Get_Library_Queue_Summary(string status_name)
Allowed status values	Pending; Under Review; Available; Collected; Unavailable
Success response	ok, status, request_count, copies_requested
Invalid response	ok = false and a readable error
Side effects	None: no data writes, emails or external API calls
Exposure	Manually tested in the owner function editor; no public API published

3. Implementation Procedure

1. Opened Workflow > Functions and created a function returning map.
2. Added status_name as a string argument.
3. Normalized null/whitespace input and checked an explicit status allow-list.
4. Used a filtered count and sum to obtain queue statistics.
5. Returned zero copies for an empty queue.
6. Saved and executed three tests in Development.

241401016

4. Deluge Source

```
map Get_Library_Queue_Summary(string status_name)
{
  result = Map();
  allowed = {"Pending","Under Review","Available","Collected","Unavailable"};
  status_value = ifnull(status_name,"").trim();
  if(!allowed.contains(status_value))
  {
    result.put("ok",false);
    result.put("error","Use a valid library request status.");
    return result;
  }
  requests = Book_Issue_Request_Form[Request_Status == status_value].count();
  copies = 0;
  if(requests > 0)
  {
    copies = Book_Issue_Request_Form[Request_Status == status_value].sum(Number_Of_Copies);
  }
  result.put("ok",true);
  result.put("status",status_value);
  result.put("request_count",requests);
  result.put("copies_requested",copies);
  return result;
}
```

5. Reuse Example

```
summary = thisapp.Get_Library_Queue_Summary("Pending");
info summary;
```

A later workflow or page can call this function to reuse the calculation. It was tested directly in the function editor and has not been wired into the existing dashboard or published to Production.

241401016

6. Test Cases and Actual Outputs

Test input	Observed output	Result
Pending	ok: true request_count: 2 copies_requested: 6	Pass
InvalidStatus	ok: false error: Use a valid library request status.	Pass
Collected	ok: true request_count: 0 copies_requested: 0	Pass

7. Test Interpretation

The Pending queue contains requests for five and one copies, giving a total of six. An invalid status returns a clear error instead of silently reporting an empty queue. The valid but empty Collected queue returns zeros.

All three executions displayed Executed Successfully. Blank input, whitespace normalization and every other allowed status were reviewed in code but were not separately executed in this test set.

8. Security and Maintenance

Only a fixed set of statuses is accepted. The result includes aggregate values and does not return member names, addresses or contact details. Keep invocation behind appropriate application permissions if it is later exposed through a page or API. Retest the allow-list when status choices change.

241401016

Implementation Evidence

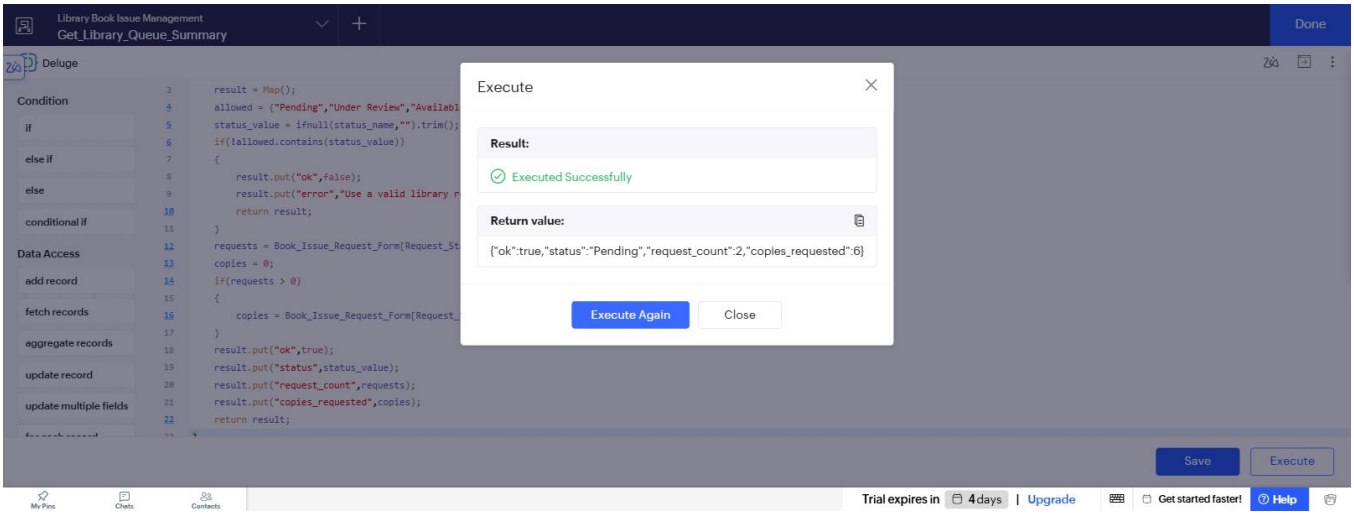


Figure 14.1. Actual Development execution for Pending: two requests and six copies.

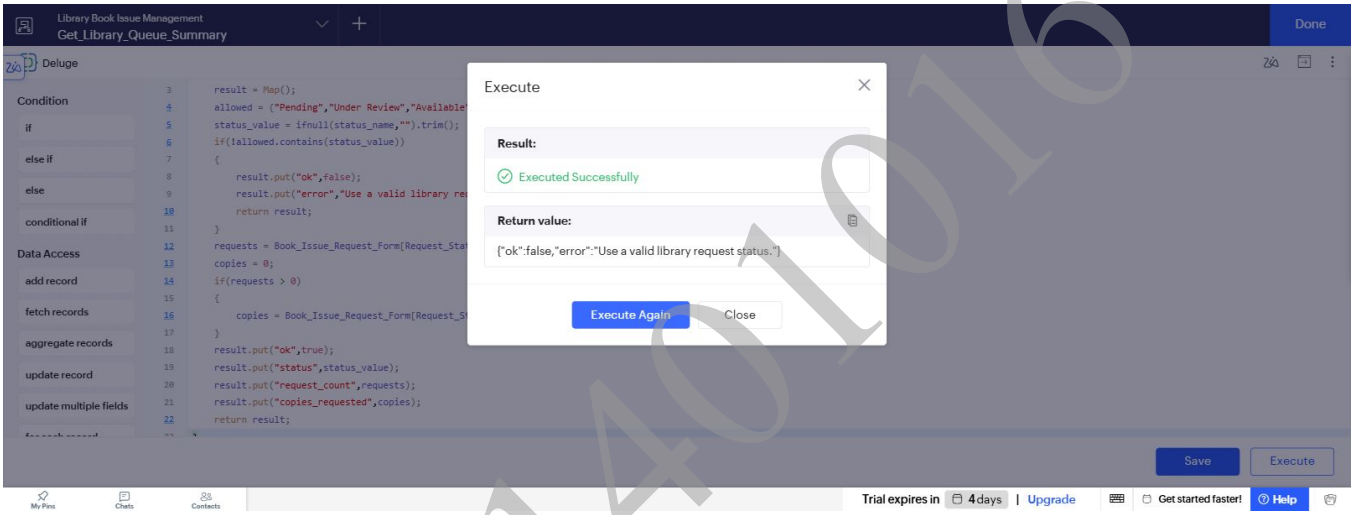


Figure 14.2. Actual Development execution for InvalidStatus: controlled validation error in the returned map.

241401016

Implementation Evidence

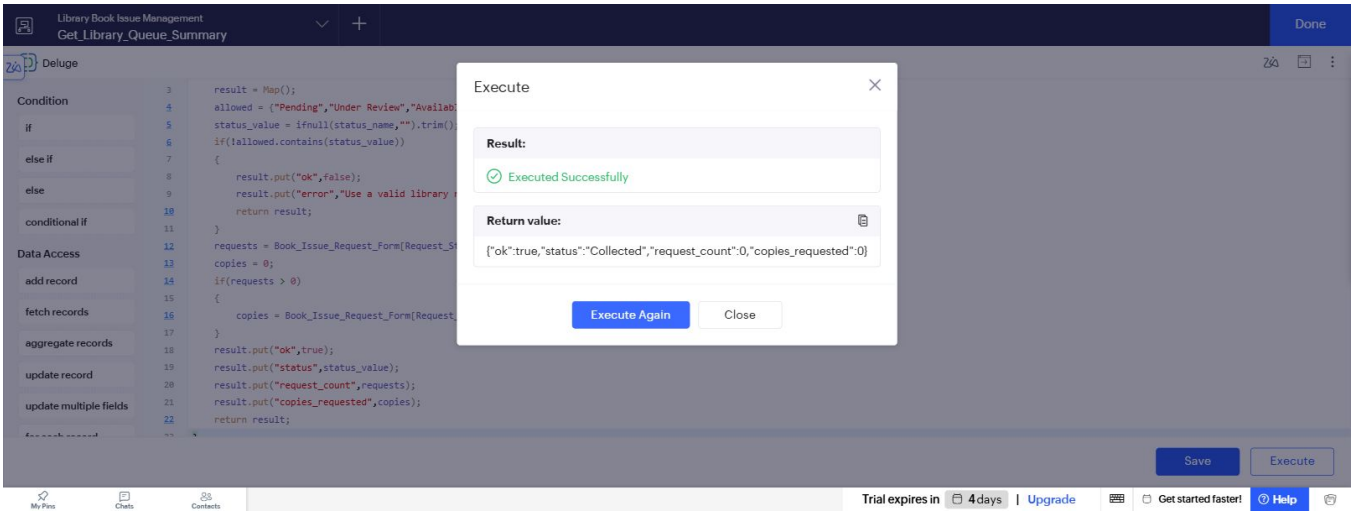


Figure 14.3. Actual Development execution for Collected: a valid empty queue returns zero requests and copies.

241401016

9. Catalyst Comparison

Custom function used here	Possible Catalyst extension
Runs inside Creator and reads the request form directly.	Would expose a defined service interface and require authorization.
Suitable for a small queue summary.	Could host a specialized computation or reservation service.
No separate hosting setup for this lab.	Requires a project, deployment and operational monitoring.

10. Result

Get_Library_Queue_Summary was implemented and all three executed test cases passed. It demonstrates how typed custom logic can extend a low-code application without rewriting the existing form and workflow.

Catalyst was studied as an alternative; no Catalyst project or cloud function was deployed. The selected custom-function route fulfills the implementation approach used in this record.

References

1. Understanding Creator functions

2. Catalyst serverless introduction

Official sources checked on 16 October 2026. App observations and test results are identified separately.

241401016